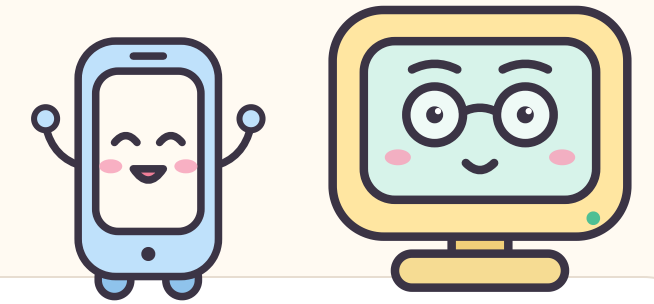




第 19 集 · 约 21 分钟

网页②：界面

不用 `innerHTML`，也能把页面写出来

19

从零写出 LanShare

这一集你会学到

1. CSP 下的页面：没有内联脚本、内联样式，也不用 `innerHTML`
2. 启动顺序：先把 K 从地址栏抹掉，再加载加密模块
3. `hashchange`：页面开着时又扫了一次码
4. http 页面没有剪贴板 API，复制要退回 `execCommand`
5. 口令登录被暂停时，页面怎么提示



小手机打开页面的第一秒



发生了什么

窃听器只看到一个普通的页面请求

页面和脚本都是公开的，不含秘密

只有证明，没有钥匙

窃听器只看到密文



页面的规矩：CSP

● 老电脑给每个页面加了 Content-Security-Policy 响应头

default-src 'none'; script-src 'self' 'wasm-unsafe-eval'; style-src 'self'; img-src 'self' blob: ...

● 不能有内联脚本、内联样式、onclick 这种内联事件

● 不用 innerHTML，只用 textContent 和 createElement

● 页面不引用任何外部网址



index.html: 头部和图标

index.html crates/lanshare/web/

第 3-22 行 · 共 22 行

```
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
6 <meta name="theme-color" content="#2563eb">
7 <title>局域网快传</title>
8 <link rel="icon" href="/favicon.ico">
9 <link rel="stylesheet" href="/style.css">
10 </head>
11 <body class="mobile">
12 <svg class="sprite" aria-hidden="true">
13   <defs>
14     <symbol id="logo" viewBox="0 0 64 64">
15       <rect x="2" y="2" width="60" height="60" rx="14" fill="#2563eb"/>
16       <path d="M23 13 L13 29 H19.5 V51 H26.5 V29 H33 Z M41 51 L51 35 H44.5 V13 H37.5 V35 H31 Z" fill="#fff"/>
17     </symbol>
18     <symbol id="i-lock" viewBox="0 0 24 24"><path d="M6 11h12v9H6zM8 11V8a4 4 0 0 1 8 0v3"/></symbol>
19     <symbol id="i-upload" viewBox="0 0 24 24"><path d="M12 19V7M7 12L5-5 5 5M5 4h14"/></symbol>
20     <symbol id="i-folder" viewBox="0 0 24 24"><path d="M3 7a2 2 0 0 1 2-2h4l2 2h8a2 2 0 0 1 2 2v8a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2z"/></symbol>
21   </defs>
22 </svg>
```





index.html: 几个状态和口令页

index.html crates/lanshare/web/

第 20-39 行 · 共 39 行

```
20     <symbol id="i-folder" viewBox="0 0 24 24"><path d="M3 7a2 2 0 0 1 2-2h4l2 2h8a2 2 0 0 1 2 2v8a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2z"/></symbol>
21 </defs>
22 </svg>
23
24 <div id="offline" class="offline-bar" hidden>连不上电脑上的局域网快传，正在重试...</div>
25 <div id="booting" class="booting">正在建立加密连接...</div>
26
27 <div class="login-wrap" id="login" hidden>
28   <section class="card login">
29     <svg class="logo"><use href="#logo"/></svg>
30     <h1>局域网快传</h1>
31     <p class="muted">用手机相机扫电脑上的二维码即可进入。<br>不方便扫码时，输入电脑上显示的 6 位口令。</p>
32     <form id="loginForm">
33       <input id="pinInput" type="text" inputmode="numeric" autocomplete="one-time-code" maxlength="6" placeholder="....." aria-label="口令">
34       <button class="btn" type="submit" id="loginBtn">进入</button>
35     </form>
36     <p id="loginMsg" class="err"></p>
37     <p class="secure-note"><svg class="i"><use href="#i-lock"/></svg>口令不会在网络上传输，抓包也看不到</p>
38   </section>
39 </div>
```



index.html: 主界面

index.html crates/lanshare/web/

第 94-113 行 • 共 113 行

```
94     </section>
95
96     <section class="card" id="texts">
97         <div class="card-head"><h2>文字</h2></div>
98         <div class="composer">
99             <textarea id="textInput" rows="2" placeholder="输入或粘贴文字，发给所有设备"></textarea>
100             <button class="btn" id="sendText" type="button">发送</button>
101         </div>
102         <ul id="textList" class="msgs"></ul>
103     </section>
104 </div>
105 </main>
106 </div>
107
108 <div id="overlay" class="overlay" hidden><div class="overlay-box">松开鼠标，发送给所有设备</div></div>
109 <div id="toast" class="toast" role="status"></div>
110
111 <script type="module" src="/app.js"></script>
112 </body>
113 </html>
```



style.css：拷贝过来，挑三段讲



style.css

crates/lanshare/web/

从仓库拷贝

第 45–64 行 • 共 178 行

```
45 @media (max-width: 420px) { .chip.status { display: none; } .chip.secure { margin-left: auto; } }
46
47 /* ----- 布局 ----- */
48 main { max-width: 1160px; margin: 0 auto; padding: 12px 20px 40px; display: grid; gap: 16px;
49   grid-template-columns: minmax(0, 1fr); }
50 .col { display: grid; gap: 16px; align-content: start; min-width: 0; }
51 body.desktop main { grid-template-columns: minmax(0, 1fr) 320px; }
52 @media (max-width: 760px) { body.desktop main { grid-template-columns: minmax(0, 1fr); } }
53 /* 手机上拆掉两栏，按“发送 → 文件 → 文字 → 连接”排 */
54 body.mobile .col { display: contents; }
55 body.mobile #upload { order: 1; }
56 body.mobile #files { order: 2; }
57 body.mobile #texts { order: 3; }
58 body.mobile #connect { order: 4; }
59 .card { background: var(--card); border-radius: var(--radius); box-shadow: var(--shadow); padding: 18px; min-width: 0; }
60 .card-head { display: flex; align-items: center; gap: 8px; margin-bottom: 14px; }
61 .card-head h2 { margin: 0; font-size: 15px; font-weight: 650; }
62 .count { color: var(--faint); font-size: 13px; }
63 .muted { color: var(--muted); font-size: 13px; }
64 .secure-note { display: flex; align-items: center; justify-content: center; gap: 6px; margin: 12px 0 0;
```



app.js: 开头和常量

● ● ● app.js crates/lanshare/web/

第 18–37 行 • 共 37 行

```
18  [["apk", "exe", "msi", "dmg", "ipa"], "app"],
19  ];
20  const ERRORS = {
21    disk_full: "电脑磁盘空间不足",
22    changed: "文件在电脑上被改动了，请刷新后重试",
23    not_found: "文件已经不存在了",
24    text_too_long: "文字太长了（最多 10000 字）",
25    empty_text: "文字不能为空",
26    too_many_uploads: "同时上传的文件太多，请稍后再试",
27  };
28
29  let client;
30  let isDesktop = false;
31  let pollTimer = null;
32  let pollCount = 0;
33  let failures = 0;
34  let lastFilesKey = "";
35  let lastTextsKey = "";
36  let busy = 0;
37  let qrUrl = null;
```



小工具：不用 innerHTML 造元素

● ● ●

app.js crates/lanshare/web/

第 74-93 行 • 共 93 行

```
74   const dot = name.lastIndexOf(".");
75   const ext = dot > 0 ? name.slice(dot + 1).toLowerCase() : "";
76   const kind = (TYPES.find(([exts]) => exts.includes(ext)) || [null, ""])[1];
77   return el("span", "badge " + kind, ext ? ext.slice(0, 4).toUpperCase() : "FILE");
78 }
79
80 let toastTimer = null;
81 function toast(text) {
82   const t = $("toast");
83   t.textContent = text;
84   t.classList.add("show");
85   clearTimeout(toastTimer);
86   toastTimer = setTimeout(() => t.classList.remove("show"), 2200);
87 }
88
89 function describe(error) {
90   if (error instanceof ServerError) return ERRORS[error.code] || "出错了 (" + error.code + ") ";
91   if (error instanceof HttpError) return "出错了 (HTTP " + error.status + ") ";
92   return "网络中断，请重试";
93 }
```



登录与进入

app.js crates/lanshare/web/

第 173–192 行 • 共 196 行

```
173
174 $("loginForm").addEventListener("submit", async (event) => {
175     event.preventDefault();
176     const pin = $("pinInput").value.trim();
177     if (!/^\d{6}$/.test(pin)) {
178         $("loginMsg").textContent = "口令是 6 位数字";
179         return;
180     }
181     $("loginBtn").disabled = true;
182     $("loginMsg").textContent = "";
183     try {
184         client.serverIdCache = null;
185         await client.pairWithPin(pin);
186         $("pinInput").value = "";
187         await enterApp();
188     } catch (e) {
189         if (e instanceof WrongPin) $("loginMsg").textContent = "口令不对，请看电脑上显示的 6 位数字";
190         else if (e instanceof HttpError && e.status === 429) $("loginMsg").textContent = "尝试太频繁，请 1 分钟后再试";
191         else if (e instanceof HttpError && e.status === 423) $("loginMsg").textContent = "有人连续输错口令太多次，口令登录已暂停。请在电脑上的主界面里重新开启，或者直接扫码";
192         else $("loginMsg").textContent = "连不上电脑上的程序，请确认它还在运行";
```



轮询

● ● ●

app.js crates/lanshare/web/

第 209-228 行 • 共 228 行

```
209
210 document.addEventListener("visibilitychange", () => {
211   if (!document.hidden && pollTimer) refresh();
212 });
213
214 async function refresh() {
215   try {
216     const [files, texts] = await Promise.all([client.rpc({ op: "list" }), client.rpc({ op: "texts" })]);
217     renderFiles(files.files);
218     renderTexts(texts.texts);
219     // 本机界面每 10 秒看一次口令状态：被暂停时要让电脑前的人看到
220     if (isDesktop && ++pollCount % 5 === 0) showPin(await client.rpc({ op: "info" }));
221     failures = 0;
222     $("offline").hidden = true;
223   } catch (e) {
224     if (e instanceof Unauthorized) return recover();
225     failures += 1;
226     if (failures >= 2) $("offline").hidden = false;
227   }
228 }
```



文件列表和下载

● ● ●

app.js crates/lanshare/web/

第 266–285 行 • 共 292 行

```
266
267 async function download(f, meta) {
268   const original = meta.textContent;
269   busy += 1;
270   try {
271     const blob = await client.downloadFile(f, {
272       onProgress: (n) => { meta.textContent = "下载中 " + Math.floor((n / Math.max(f.size, 1)) * 100) + "%"; },
273     });
274     const url = URL.createObjectURL(blob);
275     const a = el("a");
276     a.href = url;
277     a.download = f.name;
278     document.body.appendChild(a);
279     a.click();
280     a.remove();
281     setTimeout(() => URL.revokeObjectURL(url), 60_000);
282     toast("已下载 " + f.name);
283   } catch (e) {
284     if (e instanceof Unauthorized) return recover();
285     toast(describe(e));
```



文字：复制的坑

app.js

crates/lanshare/web/

第 337–356 行 • 共 366 行

```

337
338 async function sendText() {
339     const input = $("textInput");
340     const text = input.value;
341     if (!text.trim()) return;
342     const button = $("sendText");
343     button.disabled = true;
344     try {
345         const r = await client.rpc({ op: "text_add", text });
346         if (r.ok) {
347             input.value = "";
348             refresh();
349         } else {
350             toast(ERRORS[r.error] || "发送失败");
351         }
352     } catch (e) {
353         if (e instanceof Unauthorized) return recover();
354         toast("发送失败，请检查网络");
355     } finally {
356         button.disabled = false;
    
```



上传队列和拖放

app.js crates/lanshare/web/

第 445–464 行 · 共 464 行

```

445 window.addEventListener("dragleave", (event) => {
446     if (!hasFiles(event)) return;
447     dragDepth = Math.max(0, dragDepth - 1);
448     if (dragDepth === 0) $("overlay").hidden = true;
449 });
450 window.addEventListener("dragover", (event) => { if (hasFiles(event)) event.preventDefault(); });
451 window.addEventListener("drop", (event) => {
452     if (!hasFiles(event)) return;
453     event.preventDefault();
454     dragDepth = 0;
455     $("overlay").hidden = true;
456     if (!$("app").hidden && event.dataTransfer.files.length) addFiles(event.dataTransfer.files);
457 });
458
459 window.addEventListener("beforeunload", (event) => {
460     if (busy > 0 || queue.length) {
461         event.preventDefault();
462         event.returnValue = "";
463     }
464 });

```



启动：先抹掉密钥，再加载

```
app.js crates/lanshare/web/ 第 477-496 行 • 共 496 行

477   }
478   try {
479     if (key) {
480       await client.pairWithKey(key);
481       return await enterApp();
482     }
483     if (await client.resume()) return await enterApp();
484     showLogin();
485   } catch (e) {
486     showLogin(e instanceof Unauthorized ? "二维码已过期（电脑上的程序重启过），请重新扫码或输入口令"
487       : "连不上电脑上的程序，请确认它还在运行");
488   }
489 }
490
491 boot();
492
493 // 页面开着时又收到带新密钥的链接：只有 # 后面变了，浏览器不会重新加载页面。手动重新加载，按新密钥配对
494 window.addEventListener("hashchange", () => {
495   if (keyFromFragment(location.hash)) location.reload();
496 });
```



页面静态检查

page.test.mjs

web-tests/

第 17-36 行 · 共 46 行

```

17
18 test("没有内联脚本、内联样式、内联事件 (CSP 会拦掉)", () => {
19     const scriptTags = html.match(/<script\b[^\>]*>[\s\S]*?<\/script>/g) || [];
20     for (const tag of scriptTags) {
21         assert.match(tag, /\bsrc="\/[a-z]+\.\js"/, "脚本必须外链本站文件");
22         assert.match(tag, /><\/script>$/, "script 标签里不能有内容");
23     }
24     assert.doesNotMatch(html, /\sstyle\s*=/i, "不能有 style 属性");
25     assert.doesNotMatch(html, /\son[a-z]+\s*=/i, "不能有 onclick 之类的内联事件");
26     assert.doesNotMatch(html, /<style\b/i, "不能有 <style> 块");
27 });
28
29 test("不引用任何外部资源 (断网局域网也要能用)", () => {
30     for (const [name, text] of [["index.html", html], ["style.css", css], ...Object.entries(scripts)]) {
31         assert.doesNotMatch(text, /(src|href)\s*=\s*"?(https?:)?\/\//i, name);
32         assert.doesNotMatch(text, /@import|url\(\s*"?(https?:)?\/\//i, name);
33     }
34 });
35
36 test("从不把字符串当 HTML/代码执行", () => {

```

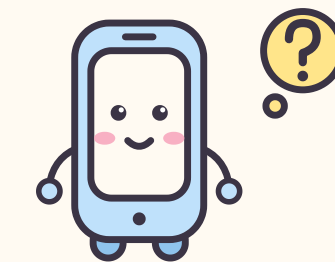


编译，跑全部 Node 测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build -p lanshare
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 18.14s

PS D:\code\lanshare> node --test "web-tests/*.test.mjs"
WASM seal 177 MiB/s, open 175 MiB/s (Node v24.12.0, 1 MiB chunks)
✓ 握手标签、会话密钥、方向密钥与 OpenSSL 一致 (3.723ms)
✓ 确认标签与 OpenSSL 一致，校验区分角色 (1.2458ms)
✓ 加密结果与 OpenSSL 逐字节一致 (含 64 位计数器) (0.9235ms)
✓ 解密：正确的能解开，任何篡改都解不开 (0.7061ms)
✓ 大块数据往返 (1 MiB + 3) (41.7504ms)
✓ PAKE：同口令同钥匙，错口令不同钥匙，句柄只能用一次 (11.4834ms)
✓ 内存不泄漏：反复加解密后线性内存不再增长 (2428.2464ms)
✓ 测速 (记入 docs/bench.md) (2915.5746ms)
✓ exe 内嵌的网页资源与仓库里的文件一致 (防止打包进旧的 WASM/页面) (181.4676ms)
✓ 二维码片段解析 (0.4178ms)
✓ 扫码配对 → 加密 RPC (15.6801ms)
✓ 上传 (分块并发) → 列表 → 下载，内容一致，进度走满 (124.893ms)
✓ 空文件与重名 (36.7924ms)
✓ 文字互发 (9.4128ms)
```



选择题

app.js 为什么在加载 WASM 之前，就先用 history.replaceState 抹掉地址栏里的密钥？

- A 因为不抹掉，WASM 加载不了
- B 不让密钥在地址栏里多停留加载 WASM 的那段时间
- C 因为 replaceState 会把密钥发给服务端



选择题

app.js 为什么在加载 WASM 之前，就先用 history.replaceState 抹掉地址栏里的密钥？

A 因为不抹掉，WASM 加载不了

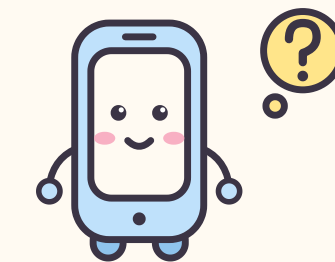
加载 WASM 和地址栏里有没有密钥无关。

C 因为 replaceState 会把密钥发给服务端

replaceState 只改浏览器本地的地址和历史记录，不发任何请求。

B 不让密钥在地址栏里多停留加载 WASM 的那段时间

先抹掉再加载，密钥在地址栏里停留的时间最短，这是安全审查提出的修改。



选择题

手机通过 http 加局域网 IP 打开页面，点“复制”时会走哪条路？

A navigator.clipboard.writeText

B 造一个看不见的文本框，选中后 execCommand copy

C 复制不了



选择题

手机通过 http 加局域网 IP 打开页面，点“复制”时会走哪条路？

A navigator.clipboard.writeText

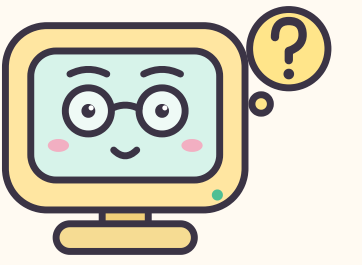
这不是安全上下文，navigator.clipboard 根本不存在。

C 复制不了

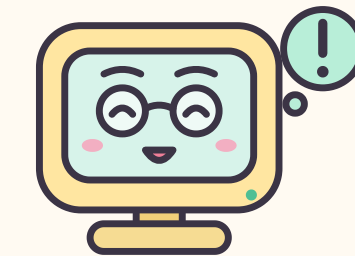
老办法在手机浏览器上仍然可用。

B 造一个看不见的文本框，选中后 execCommand copy

http 加局域网 IP 不是安全上下文，没有剪贴板 API，只能退回 execCommand 的老办法。



文字消息是别人发来的。如果用 `innerHTML` 把它插进列表，会发生什么？我们怎么防的？



想一想

文字消息是别人发来的。如果用 innerHTML 把它插进列表，会发生什么？我们怎么防的？

捣蛋鬼可以发一条文字消息，内容是一段 HTML，比如一个带 onerror 的 img 标签。用 innerHTML 插进去，浏览器会把它当 HTML 解析，就可能在所有打开页面的设备上执行脚本，偷走密钥。我们防了三层：服务端清理文件名，尖括号进不来；页面从不用 innerHTML，文字一律用 textContent；CSP 不允许内联脚本和内联事件，就算哪天漏了一处，浏览器也会拦下。页面测试守着第二层。



这一集写完了

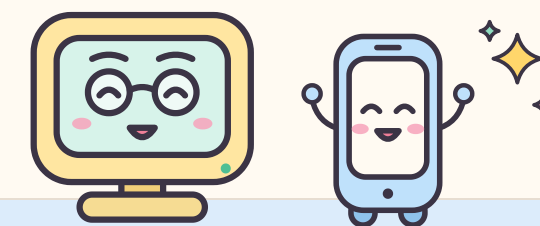
```
lanshare/  
├── crates/  
│   └── lanshare/  
│       ├── src/ 19 个文件  
│       ├── tests/ 5 个文件  
│       └── web/  
│           ├── app.js 这一集写的  
│           ├── icon.ico 从仓库拷贝  
│           ├── index.html 这一集写的  
│           ├── lanshare.wasm 从仓库拷贝  
│           └── proto.js
```

```
├── style.css 这一集写的  
├── build.rs  
├── Cargo.toml  
├── lanshare-proto/ 13 个文件  
├── lanshare-wasm/ 2 个文件  
├── web-tests/  
│   ├── crypto.test.mjs  
│   ├── gen-vectors.mjs  
│   ├── interop.test.mjs  
│   └── page.test.mjs 这一集写的  
└── Cargo.toml
```



这一集写了什么

- ✓ CSP：没有内联脚本和样式；从不用 innerHTML，文字一律textContent
- ✓ 二维码 SVG 包成 Blob，用 img 显示
- ✓ 启动先抹掉地址栏里的 K，再加载 WASM；监听 hashchange 处理重新扫码
- ✓ http 页面没有剪贴板 API，复制退回 execCommand
- ✓ 口令登录暂停：本机界面显示提示和“重新开启”，手机上提示去电脑上开启或扫码



下一集

全课完结，感谢收看